2.1: Various pointer

1. Basic Pointer

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p = &a;

    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", &a);
    printf("Pointer value = %p\n", p);
    printf("Value using pointer = %d\n", *p);

    return 0;
}
```

2. NULL Pointer

```
#include <stdio.h>

int main() {
    int *p = NULL;

    if (p == NULL)
        printf("Pointer is NULL.\n");
    else
        printf("%d", *p);

    return 0;
}
```

3. Wild Pointer

```
#include <stdio.h>

int main() {
    int *p;   // Wild pointer

    printf("Pointer declared but not initialized.\n");

    return 0;
}
```

4. Dangling Pointer

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int *)malloc(sizeof(int));

    *p = 100;

    printf("Value = %d\n", *p);

    free(p);

    p = NULL;      // Prevent dangling pointer
```

```
    return 0;
}
```

5. Pointer Arithmetic

```
#include <stdio.h>

int main() {
    int arr[] = {10,20,30,40,50};

    int *p = arr;

    for(int i=0;i<5;i++)
        printf("%d ", *(p+i));

    return 0;
}
```

6. Pointer to Pointer

```
#include <stdio.h>

int main() {
    int a = 50;

    int *p = &a;
    int **q = &p;

    printf("%d\n", a);
    printf("%d\n", *p);
    printf("%d\n", **q);

    return 0;
}
```

7. Void Pointer

```
#include <stdio.h>

int main() {
    int a = 25;

    void *vp = &a;

    printf("%d\n", *(int *)vp);

    return 0;
}
```

8. Constant Pointer

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    int *const p = &a;
```

```
    *p = 50;

    printf("%d\n", *p);

    // p = &b; // Not allowed

    return 0;
}
```

9. Pointer to Constant

```
#include <stdio.h>

int main() {
    int a = 30;
    int b = 40;

    const int *p = &a;

    p = &b;

    printf("%d\n", *p);

    return 0;
}
```

10. Constant Pointer to Constant

```
#include <stdio.h>

int main() {
    int a = 100;

    const int *const p = &a;

    printf("%d\n", *p);

    return 0;
}
```

11. Function Pointer

```
#include <stdio.h>

int add(int x, int y)
{
    return x + y;
}

int main()
{
    int (*fp)(int,int);

    fp = add;

    printf("Sum = %d\n", fp(10,20));

    return 0;
```

```
}
```

12. Pointer with Dynamic Memory (malloc)

```
#include <stdio.h>
#include <stdlib.h>

int main() {

   int *p;

   p = (int *)malloc(sizeof(int));

   *p = 75;

   printf("%d\n", *p);

   free(p);

   return 0;
}
```

13. Array of Pointers

```
#include <stdio.h>

int main() {

   int a=10,b=20,c=30;

   int *arr[3];

   arr[0]=&a;
   arr[1]=&b;
   arr[2]=&c;

   for(int i=0;i<3;i++)
      printf("%d ", *arr[i]);

   return 0;
}
```

14. Pointer to Array

```
#include <stdio.h>

int main() {

   int arr[5]={1,2,3,4,5};

   int (*p)[5]=&arr;

   for(int i=0;i<5;i++)
      printf("%d ", (*p)[i]);

   return 0;
}
```

2.2: Program: Allocate Memory for Structure Using malloc

```
#include <stdio.h>
#include <stdlib.h>

// Structure definition
struct Student {
    int rollNo;
    char name[50];
    float marks;
};

int main() {
    // Pointer to structure
    struct Student *ptr;

    // Dynamically allocate memory for one structure
    ptr = (struct Student *)malloc(sizeof(struct Student));

    // Check if memory allocation is successful
    if (ptr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // Input values
    printf("Enter Roll Number: ");
    scanf("%d", &ptr->rollNo);

    printf("Enter Name: ");
    scanf("%s", ptr->name);

    printf("Enter Marks: ");
    scanf("%f", &ptr->marks);

    // Display values
    printf("\nStudent Details\n");
    printf("-----------------\n");
    printf("Roll Number : %d\n", ptr->rollNo);
    printf("Name        : %s\n", ptr->name);
    printf("Marks       : %.2f\n", ptr->marks);

    // Free allocated memory
    free(ptr);

    return 0;
}
```

2.3: Dynamic Memory Allocation for an Array of Structures Using malloc()

```
#include <stdio.h>
#include <stdlib.h>

struct Student
{
   int rollNo;
   char name[50];
   float marks;
};

int main()
{
   int n, i;

   struct Student *ptr;

   printf("Enter the number of students: ");
   scanf("%d", &n);

   // Dynamic memory allocation for array of structures
   ptr = (struct Student *)malloc(n * sizeof(struct Student));

   if (ptr == NULL)
   {
      printf("Memory allocation failed!\n");
      return 1;
   }

   printf("\nEnter Student Details\n");

   for(i = 0; i < n; i++)
   {
      printf("\nStudent %d\n", i + 1);

      printf("Roll Number : ");
      scanf("%d", &ptr[i].rollNo);

      printf("Name        : ");
      scanf("%s", ptr[i].name);

      printf("Marks       : ");
      scanf("%f", &ptr[i].marks);
   }

   printf("\n================================");
   printf("\n      STUDENT DETAILS");
   printf("\n================================\n");

   for(i = 0; i < n; i++)
   {
      printf("\nStudent %d\n", i + 1);
      printf("Roll Number : %d\n", ptr[i].rollNo);
```

```
        printf("Name        : %s\n", ptr[i].name);
        printf("Marks       : %.2f\n", ptr[i].marks);
    }

    free(ptr);

    printf("\nMemory released successfully.\n");

    return 0;
}
```

Experiment2.4: Dynamic Jagged Array (Rows of Different Sizes)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int **arr;
   int *cols;
   int rows;
   int i, j;

   printf("Enter the number of rows: ");
   scanf("%d", &rows);

   /* Allocate memory for row pointers */
   arr = (int **)malloc(rows * sizeof(int *));

   /* Allocate memory to store the size of each row */
   cols = (int *)malloc(rows * sizeof(int));

   if (arr == NULL || cols == NULL)
   {
      printf("Memory allocation failed.\n");
      return 1;
   }

   /* Read the size of each row and allocate memory */
   for (i = 0; i < rows; i++)
   {
      printf("Enter number of elements in Row %d: ", i + 1);
      scanf("%d", &cols[i]);

      arr[i] = (int *)malloc(cols[i] * sizeof(int));

      if (arr[i] == NULL)
      {
         printf("Memory allocation failed for Row %d.\n", i + 1);
         return 1;
      }
   }

   /* Read elements */
   printf("\nEnter the elements:\n");

   for (i = 0; i < rows; i++)
   {
      printf("Row %d:\n", i + 1);

      for (j = 0; j < cols[i]; j++)
      {
         scanf("%d", &arr[i][j]);
      }
   }
```

```
    /* Display the jagged array */
    printf("\nJagged Array:\n");

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols[i]; j++)
        {
            printf("%4d", arr[i][j]);
        }
        printf("\n");
    }

    /* Free allocated memory */
    for (i = 0; i < rows; i++)
    {
        free(arr[i]);
    }

    free(arr);
    free(cols);

    printf("\nMemory released successfully.\n");

    return 0;
}
```

Experiment 2.5: Mini Address Book using Dynamic Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Contact
{
   char name[50];
   char phone[15];
   char email[50];
};

int main()
{
   struct Contact *book = NULL;
   int count = 0;
   int choice, i, found;

   char searchName[50];

   while (1)
   {
      printf("\n========================================\n");
      printf("       MINI ADDRESS BOOK\n");
      printf("========================================\n");
      printf("1. Add Contact\n");
      printf("2. Display Contacts\n");
      printf("3. Search Contact\n");
      printf("4. Update Contact\n");
      printf("5. Delete Contact\n");
      printf("6. Exit\n");

      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
         case 1:

            book = (struct Contact *)realloc(book,
                  (count + 1) * sizeof(struct Contact));

            if (book == NULL)
            {
               printf("Memory Allocation Failed!\n");
               exit(1);
            }

            printf("\nEnter Name : ");
            scanf("%s", book[count].name);

            printf("Enter Phone : ");
            scanf("%s", book[count].phone);
```

```
    printf("Enter Email : ");
    scanf("%s", book[count].email);

    count++;

    printf("Contact Added Successfully.\n");
    break;

case 2:

    if (count == 0)
    {
       printf("Address Book is Empty.\n");
       break;
    }

    printf("\n-----------------------------------------------------------\n");
    printf("Name\t\tPhone\t\tEmail\n");
    printf("-----------------------------------------------------------\n");

    for (i = 0; i < count; i++)
    {
       printf("%-15s %-15s %-20s\n",
            book[i].name,
            book[i].phone,
            book[i].email);
    }

    break;

case 3:

    if (count == 0)
    {
       printf("Address Book is Empty.\n");
       break;
    }

    printf("Enter Name to Search: ");
    scanf("%s", searchName);

    found = 0;

    for (i = 0; i < count; i++)
    {
       if (strcmp(book[i].name, searchName) == 0)
       {
          printf("\nContact Found\n");
          printf("Name  : %s\n", book[i].name);
          printf("Phone : %s\n", book[i].phone);
          printf("Email : %s\n", book[i].email);

          found = 1;
          break;
       }
```

```
    }

    if (!found)
        printf("Contact Not Found.\n");

    break;

case 4:

    if (count == 0)
    {
        printf("Address Book is Empty.\n");
        break;
    }

    printf("Enter Name to Update: ");
    scanf("%s", searchName);

    found = 0;

    for (i = 0; i < count; i++)
    {
        if (strcmp(book[i].name, searchName) == 0)
        {
            printf("Enter New Phone : ");
            scanf("%s", book[i].phone);

            printf("Enter New Email : ");
            scanf("%s", book[i].email);

            printf("Contact Updated Successfully.\n");

            found = 1;
            break;
        }
    }

    if (!found)
        printf("Contact Not Found.\n");

    break;

case 5:

    if (count == 0)
    {
        printf("Address Book is Empty.\n");
        break;
    }

    printf("Enter Name to Delete: ");
    scanf("%s", searchName);

    found = 0;
```

```
            for (i = 0; i < count; i++)
            {
               if (strcmp(book[i].name, searchName) == 0)
               {
                  found = 1;

                  for (int j = i; j < count - 1; j++)
                  {
                     book[j] = book[j + 1];
                  }

                  count--;

                  book = (struct Contact *)realloc(book,
                        count * sizeof(struct Contact));

                  printf("Contact Deleted Successfully.\n");

                  break;
               }
            }

            if (!found)
               printf("Contact Not Found.\n");

            break;

         case 6:

            free(book);

            printf("Memory Released Successfully.\n");
            printf("Exiting Program...\n");

            return 0;

         default:

            printf("Invalid Choice.\n");
      }
   }

   return 0;
}
```

Experiment 2.6: Dynamic Voting System using Dynamic Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Candidate
{
   int id;
   char name[50];
   int votes;
};

int main()
{
   struct Candidate *candidate = NULL;
   int count = 0;
   int choice;
   int i, id;
   int found;
   int maxVotes, winnerIndex;

   while (1)
   {
      printf("\n=======================================\n");
      printf("      DYNAMIC VOTING SYSTEM\n");
      printf("=======================================\n");
      printf("1. Add Candidate\n");
      printf("2. Cast Vote\n");
      printf("3. Display Voting Results\n");
      printf("4. Search Candidate\n");
      printf("5. Declare Winner\n");
      printf("6. Exit\n");

      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
         case 1:

            candidate = (struct Candidate *)realloc(candidate,
                  (count + 1) * sizeof(struct Candidate));

            if (candidate == NULL)
            {
               printf("Memory Allocation Failed!\n");
               exit(1);
            }

            printf("Enter Candidate ID : ");
            scanf("%d", &candidate[count].id);

            printf("Enter Candidate Name : ");
            scanf("%s", candidate[count].name);
```

```
    candidate[count].votes = 0;

    count++;

    printf("Candidate Added Successfully.\n");
    break;

case 2:

    if (count == 0)
    {
        printf("No Candidates Available.\n");
        break;
    }

    printf("Enter Candidate ID to Vote : ");
    scanf("%d", &id);

    found = 0;

    for (i = 0; i < count; i++)
    {
        if (candidate[i].id == id)
        {
            candidate[i].votes++;
            printf("Vote Cast Successfully.\n");
            found = 1;
            break;
        }
    }

    if (!found)
        printf("Candidate Not Found.\n");

    break;

case 3:

    if (count == 0)
    {
        printf("No Candidates Available.\n");
        break;
    }

    printf("\n-----------------------------------------\n");
    printf("ID\tCandidate\tVotes\n");
    printf("-----------------------------------------\n");

    for (i = 0; i < count; i++)
    {
        printf("%d\t%-15s%d\n",
               candidate[i].id,
               candidate[i].name,
               candidate[i].votes);
```

```
    }

    break;

case 4:

    if (count == 0)
    {
        printf("No Candidates Available.\n");
        break;
    }

    printf("Enter Candidate ID to Search : ");
    scanf("%d", &id);

    found = 0;

    for (i = 0; i < count; i++)
    {
        if (candidate[i].id == id)
        {
            printf("\nCandidate Found\n");
            printf("ID    : %d\n", candidate[i].id);
            printf("Name  : %s\n", candidate[i].name);
            printf("Votes : %d\n", candidate[i].votes);

            found = 1;
            break;
        }
    }

    if (!found)
        printf("Candidate Not Found.\n");

    break;

case 5:

    if (count == 0)
    {
        printf("No Candidates Available.\n");
        break;
    }

    maxVotes = candidate[0].votes;
    winnerIndex = 0;

    for (i = 1; i < count; i++)
    {
        if (candidate[i].votes > maxVotes)
        {
            maxVotes = candidate[i].votes;
            winnerIndex = i;
        }
    }
```

```
            printf("\n******** Election Result ********\n");
            printf("Winner : %s\n", candidate[winnerIndex].name);
            printf("Votes  : %d\n", candidate[winnerIndex].votes);

            break;

        case 6:

            free(candidate);

            printf("Memory Released Successfully.\n");
            printf("Thank You!\n");

            return 0;

        default:

            printf("Invalid Choice.\n");
      }
   }

   return 0;
}
```

Additional Experiment: Inventory Management System using Dynamic Structures

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Product
{
   int id;
   char name[30];
   int quantity;
   float price;
};

int main()
{
   struct Product *item = NULL;
   int n = 0;
   int choice;
   int i, pos, id;

   while (1)
   {
      printf("\n========== INVENTORY MANAGEMENT ==========\n");
      printf("1. Add Product\n");
      printf("2. Display Products\n");
      printf("3. Search Product\n");
      printf("4. Update Product\n");
      printf("5. Delete Product\n");
      printf("6. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
         case 1:

            item = (struct Product *)realloc(item,
                  (n + 1) * sizeof(struct Product));

            if (item == NULL)
            {
               printf("Memory Allocation Failed!\n");
               exit(0);
            }

            printf("\nEnter Product ID: ");
            scanf("%d", &item[n].id);

            printf("Enter Product Name: ");
            scanf("%s", item[n].name);

            printf("Enter Quantity: ");
            scanf("%d", &item[n].quantity);
```

```
    printf("Enter Price: ");
    scanf("%f", &item[n].price);

    n++;

    printf("Product Added Successfully.\n");
    break;

case 2:

    if (n == 0)
    {
        printf("Inventory is Empty.\n");
        break;
    }

    printf("\n-----------------------------------------------------\n");
    printf("ID\tName\tQuantity\tPrice\n");
    printf("-----------------------------------------------------\n");

    for (i = 0; i < n; i++)
    {
        printf("%d\t%s\t%d\t\t%.2f\n",
               item[i].id,
               item[i].name,
               item[i].quantity,
               item[i].price);
    }

    break;

case 3:

    printf("Enter Product ID to Search: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++)
    {
        if (item[i].id == id)
        {
            printf("\nProduct Found\n");
            printf("ID : %d\n", item[i].id);
            printf("Name : %s\n", item[i].name);
            printf("Quantity : %d\n", item[i].quantity);
            printf("Price : %.2f\n", item[i].price);
            break;
        }
    }

    if (i == n)
        printf("Product Not Found.\n");

    break;
```

```
case 4:

    printf("Enter Product ID to Update: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++)
    {
        if (item[i].id == id)
        {
            printf("Enter New Name: ");
            scanf("%s", item[i].name);

            printf("Enter New Quantity: ");
            scanf("%d", &item[i].quantity);

            printf("Enter New Price: ");
            scanf("%f", &item[i].price);

            printf("Product Updated Successfully.\n");
            break;
        }
    }

    if (i == n)
        printf("Product Not Found.\n");

    break;

case 5:

    printf("Enter Product ID to Delete: ");
    scanf("%d", &id);

    pos = -1;

    for (i = 0; i < n; i++)
    {
        if (item[i].id == id)
        {
            pos = i;
            break;
        }
    }

    if (pos == -1)
    {
        printf("Product Not Found.\n");
    }
    else
    {
        for (i = pos; i < n - 1; i++)
        {
            item[i] = item[i + 1];
        }
```

```
            n--;

            item = (struct Product *)realloc(item,
                  n * sizeof(struct Product));

            printf("Product Deleted Successfully.\n");
         }

         break;

      case 6:

         free(item);

         printf("Memory Released.\n");
         printf("Program Terminated.\n");

         return 0;

      default:

         printf("Invalid Choice.\n");
   }
  }

  return 0;
}
```